

Praktikum 6: Hashtabellen

Die vollständige Implementierung (**HashTableOpenAddressing**) finden Sie in `vorlesung/L07_hashtable/`. Alle Aufgaben bauen darauf auf — Sie implementieren Erweiterungen oder analysieren das vorhandene Verhalten. Nutzen Sie die in `analyze_hashtable.py` definierte Hash- und Sondierungsfunktion.

1. Implementieren Sie eine Klasse **HashTableChaining**, die Kollisionen durch Verkettung auflöst, analog zu **HashTableOpenAddressing**. Jeder Slot enthält eine Python-Liste der gespeicherten Schlüssel. Bieten Sie folgende Methoden an:
 - a) `insert(x)`: Fügt `x` ein. Bereits vorhandene Werte werden nicht doppelt eingefügt.
 - b) `search(x)`: Gibt `True` zurück, falls `x` enthalten ist, sonst `False`.
 - c) `delete(x)`: Entfernt `x` aus der Liste des zugehörigen Slots, falls vorhanden.
 - d) `__str__()`: Gibt alle Slots mit ihren Ketten aus (auch leere Slots).
 - e) `alpha()`: Belegungsfaktor n/m (n = Gesamtanzahl gespeicherter Elemente über alle Listen).
2. Überprüfen Sie Ihre Implementierung, indem Sie eine Hashtabelle der Größe 20 anlegen und `seq0.txt` einfügen. Nutzen Sie die bekannte Hashfunktion.
 - a) Welchen Belegungsfaktor hat die Tabelle nach dem Einfügen?
 - b) Löschen Sie Eintrag 52 und überprüfen Sie mit `__str__`, ob die Liste des Slots geleert wurde.
 - c) Fügen Sie anschließend erneut `seq0.txt` ein. Überschreitet der Belegungsfaktor dabei 1? Kann `insert` bei der Verkettung je `False` zurückgeben?
 - d) Erklären Sie: Warum wird bei der Verkettung kein `DELETED_MARK` benötigt?
3. Legen Sie eine **HashTableOpenAddressing** der Größe 20 an und fügen Sie `seq0.txt` ein. Nutzen Sie die bekannten Hash- und Sondierungsfunktionen.
 - a) Löschen Sie den Eintrag 52. Was gibt `__str__` für den betroffenen Slot aus?
 - b) Betrachten Sie den Quellcode von `search`. Warum wird die Suche bei `DELETED` fortgesetzt, aber bei `UNUSED` abgebrochen?
 - c) Angenommen, `delete` würde die Zelle auf `UNUSED` setzen: Welche Operation würde dadurch fehlerhaft? Konstruieren Sie ein konkretes Beispiel.
 - d) Fügen Sie nach dem Löschen von 52 den Wert 24 ein. Berechnen Sie manuell (mit `h` und `f` aus `analyze_hashtable.py`), wo 24 landet, und verifizieren Sie dies mit `__str__`.
4. Legen Sie eine **HashTableOpenAddressing** der Größe 90 an und fügen Sie alle 100 Werte aus `seq1.txt` ein.
 - a) Wie viele Werte werden trotz freier Slots abgewiesen? Warum versagt die Sondierungsfunktion `f` bei dieser Tabellengröße?
 - b) Wiederholen Sie den Versuch mit Größe 89. Können mehr Werte aufgenommen werden? Erklären Sie, warum eine Primzahl als Tabellengröße im Vorteil ist.
 - c) Wiederholen Sie den Versuch mit Ihrer **HashTableChaining** (Größe 20, `seq1.txt`). Wie viele Werte werden aufgenommen? Wie groß ist der Belegungsfaktor?
 - d) Die theoretische mittlere Sondierungsanzahl bei offener Adressierung beträgt $1 / (1 - \alpha)$. Berechnen Sie diesen Wert für $\alpha = 0,5$ und $\alpha = 0,9$. Bis zu welchem α ist offene Adressierung in der Praxis noch akzeptabel?

5. Messen Sie für **HashTableOpenAddressing** (mit f) und **HashTableChaining** (mit h) die Anzahl der Vergleiche (`ctx.comparisons`) beim Suchen aller eingefügten Werte. Befüllen Sie Hashtabellen der Größen [50, 100, 200, 500, 1000] bis zu einem Belegungsfaktor von ca. 0,7 mit zufälligen Werten (`Array.random`) und stellen Sie die Ergebnisse in einem Plot dar.
- a) Welche Strategie erfordert bei gleichem α mehr Vergleiche?
 - b) Erhöhen Sie den Belegungsfaktor auf 0,9. Wie verändert sich das Bild?
 - c) Nennen Sie je einen Vorteil der Verkettung und der offenen Adressierung hinsichtlich Cache-Verhalten, Speicherverbrauch oder Parallelisierbarkeit.